WEB_102 PRACTICAL_2 REPORT

1SWE

Chimi rinzin

02250346

25/3/2026

AIM:

- Creating RESTful API endpoints by adhering to URI design and Implementing API endpoints with proper HTTP methods and status codes

THEORY:

RESTful API design emphasizes the development of straightforward and significant endpoints by employing an appropriate URI (Uniform Resource Identifier) structure. URIs must depict resources with nouns instead of verbs, for instance, /practicals or /users, and must adhere to a uniform and structured format like /practicals/{id}/users. This enhances clarity and simplifies the API for better comprehension and utilization. Effective URI design minimizes unnecessary complications and guarantees that resources are systematically arranged.

In REST APIs, HTTP methods specify actions on resources, including GET for data retrieval, POST for resource creation, PUT for updates, and DELETE for their removal. In addition to these techniques, suitable HTTP status codes are applied to show the outcome of requests, like 200 for success, 201 for creation, 404 for not found, and 500 for server errors. Moreover, APIs utilize MIME types such as application/Json or application/xml for content negotiation, enabling clients and servers to share data in appropriate formats through headers like Accept and Content-Type.

Correct management of requests and responses is crucial for a dependable API. This involves verifying input data, generating structured and uniform responses (typically in JSON format), and offering helpful error messages. It is also essential to keep accurate records of API endpoints, which should include their URIs, methods, parameters, and response formats. Instruments such as Swagger or Postman assist in documenting and testing APIs, guaranteeing that developers can readily comprehend and engage with the system.

IMPLEMENTATION OF STEPS:

STEP 1: Open Vs-code (Terminal) and create a new directory for your project:

- mkdir social-media-api
- cd social-media-api 2.
- npm init -y 3.
- npm install express morgan cors helmet
- npm install nodemon --save-dev

STEP 2: Create folder structure:

- mkdir controllers

- mkdir routes
- mkdir middleware
- mkdir utils

Create Files:

- New-Item server.js
- New-Item .env
- New-Item .gitignore

STEP 3: Click **.env** file and put:

- PORT=3000

Click .**gitignore** file and put:

```
node_modules
.env
.DS_Store
```

Tap **package.json** with this start script:

-

```
"name": "social-media-api",
"version": "1.0.0",
"description": "",
"main": "index.js",
▷Debug
"scripts": {
"start": "node server.js",
"dev": "nodemon server.js"
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
"cors": "^2.8.6",
"dotenv": "^17.3.1",
"express": "^5.2.1",
"helmet": "^8.1.0",
"morgan": "^1.10.1"
},
"devDependencies": {
"nodemon": "^3.1.14"
}
}
```

STEP 4: create a basic Express server setup in server.js file:

```
const express = require('express');
const dotenv  = require('dotenv');
const morgan  = require('morgan');
const cors    = require('cors');
const helmet  = require('helmet');
const path    = require('path');

dotenv.config();
const app = express();

app.use(express.json());
app.use(morgan('dev'));
app.use(helmet());
app.use(cors());
app.use(require('./middleware/formatResponse'));

app.use(express.static('public'));
app.get('/api-docs', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'docs.html'));
});

app.use('/users', require('./routes/users'));
app.use('/posts', require('./routes/posts'));

app.get('/', (req, res) => res.json({ message: 'Welcome to Social Media API' }));

app.use(require('./middleware/errorHandler'));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running in development mode on port ${PORT}`);
});

process.on('unhandledRejection', (err) => {
  console.log(`Error: ${err.message}`);
  process.exit(1);
});
```

STEP 5: Create a new file in utils → mockData.js:

```
const users = [
  {
    id: '1',
    username: 'traveler',
    email: 'traveler@example.com',
    password: 'password123',
    full_name: 'Karma',
    profile_picture: 'https://example.com/profiles/traveler.jpg',
    bio: 'Travel photographer',
    created_at: '2023-01-15'
  },
  {
    id: '2',
    username: 'foodie',
    email: 'foodie@example.com',
    password: 'password123',
    full_name: 'Alex Johnson',
    profile_picture: 'https://example.com/profiles/foodie.jpg',
    bio: 'Food lover and chef',
    created_at: '2023-02-20'
  },
  {
    id: '3',
    username: 'fitness_guru',
    email: 'fitness@example.com',
    password: 'password123',
    full_name: 'Sam Wilson',
    profile_picture: 'https://example.com/profiles/fitness.jpg',
    bio: 'Fitness coach and wellness advocate',
    created_at: '2023-03-10'
  }
];

const posts = [
  {
    id: '1',
    caption: 'Beautiful sunset in Bali!',
    image: 'https://example.com/posts/bali-sunset.jpg',
    user_id: '1',
    created_at: '2023-03-01'
  },
```

```
const posts = [
  {
    id: '2',
    caption: 'Homemade pasta from scratch',
    image: 'https://example.com/posts/pasta.jpg',
    user_id: '2',
    created_at: '2023-03-05'
  },
  {
    id: '3',
    caption: 'Morning workout complete!',
    image: 'https://example.com/posts/workout.jpg',
    user_id: '3',
    created_at: '2023-03-08'
  },
  {
    id: '4',
    caption: 'Exploring the temples of Kyoto',
    image: 'https://example.com/posts/kyoto.jpg',
    user_id: '1',
    created_at: '2023-03-12'
  }
];

const comments = [
  {
    id: '1',
    text: 'Stunning view! Where exactly was this?',
    user_id: '2',
    post_id: '1',
    created_at: '2023-03-02'
  },
  {
    id: '2',
    text: 'I need to visit Bali someday!',
    user_id: '3',
    post_id: '1',
    created_at: '2023-03-02'
  },
```

```
  id: '3',
  text: 'That looks delicious! Recipe please?',
  user_id: '1',
  post_id: '2',
  created_at: '2023-03-06'
},
{
  id: '4',
  text: 'Goals! What is your workout routine?',
  user_id: '2',
  post_id: '3',
  created_at: '2023-03-09'
}
]

onst likes = [
{
  id: '1',
  user_id: '2',
  post_id: '1',
  created_at: '2023-03-02'
},
{
  id: '2',
  user_id: '3',
  post_id: '1',
  created_at: '2023-03-03'
},
{
  id: '3',
  user_id: '1',
  post_id: '2',
  created_at: '2023-03-06'
},
```

```
  {
    id: '4',
    user_id: '3',
    post_id: '2',
    created_at: '2023-03-07'
  }
];

const followers = [
  {
    id: '1',
    follower_id: '2',
    following_id: '1',
    created_at: '2023-02-25'
  },
  {
    id: '2',
    follower_id: '3',
    following_id: '1',
    created_at: '2023-03-01'
  },
  {
    id: '3',
    follower_id: '1',
    following_id: '2',
    created_at: '2023-03-05'
  }
];

module.exports = { users, posts, comments, likes, followers };
```

STEP 6: Create a new file in middleware → errorHandler.js:

```
const ErrorResponse = require('../utils/errorResponse');

const errorHandler = (err, req, res, next) => {
  let error = { ...err };

  error.message = err.message;

  // Log to console for dev
  console.log(err);

  res.status(error.statusCode || 500).json({
    success: false,
    error: error.message || 'Server Error'
  });
};

module.exports = errorHandler;
```

STEP 7: Create a new file in utils → errorResponse.js:

```
// utils/errorResponse.js
class ErrorResponse extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

module.exports = ErrorResponse;
```

STEP 8: Create an async handler utility in middleware → async.js:

```
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

module.exports = asyncHandler;
```

STEP 9: Create a new file in (controllers → userController.js):

```
const ErrorResponse = require('../utils/errorResponse');
const asyncHandler = require('../middleware/async');
const { users } = require('../utils/mockData');

// @desc    Get all users
// @route   GET /api/users
// @access  Public
exports.getUsers = asyncHandler(async (req, res, next) => {
  // Pagination
  const page = parseInt(req.query.page, 10) || 1;
  const limit = parseInt(req.query.limit, 10) || 10;
  const startIndex = (page - 1) * limit;
  const endIndex = page * limit;
  const total = users.length;

  // Get paginated results
  const results = users.slice(startIndex, endIndex);

  // Pagination result
  const pagination = {};

  if (endIndex < total) {
    pagination.next = {
      page: page + 1,
      limit
    };
  }

  if (startIndex > 0) {
    pagination.prev = {
      page: page - 1,
      limit
    };
  }

  res.status(200).json({
    success: true,
    count: results.length,
    page,
    total_pages: Math.ceil(total / limit),
    pagination,
    data: results
  });
});
```

```
exports.getUser = asyncHandler(async (req, res, next) => {
  const user = users.find(user => user.id === req.params.id);

  if (!user) {
    return next(
      new ErrorResponse(`User not found with id of ${req.params.id}`, 404)
    );
  }

  res.status(200).json({
    success: true,
    data: user
  });
});

// @desc Create new user
// @route POST /api/users
// @access Public

exports.createUser = asyncHandler(async (req, res, next) => {
  const newUser = {
    id: (users.length + 1).toString(),
    username: req.body.username,
    email: req.body.email,
    full_name: req.body.full_name,
    profile_picture: req.body.profile_picture || 'default-profile.jpg',
    bio: req.body.bio || '',
    created_at: new Date().toISOString().slice(0, 10)
  };

  // Check if username already exists
  const existingUser = users.find(user => user.username === newUser.username);

  if (existingUser) {
    return next(new ErrorResponse('Username already exists', 400));
  }

  users.push(newUser);

  res.status(201).json({
    success: true,
    data: newUser
  });
});
```

```
exports.updateUser = asyncHandler(async (req, res, next) => {
  let user = users.find(user => user.id === req.params.id);
  if (!user) {
    return next(
      new ErrorResponse(`User not found with id of ${req.params.id}`, 404)
    );
  }

  // Update user
  const index = users.findIndex(user => user.id === req.params.id);

  users[index] = {
    ...user,
    ...req.body,
    id: user.id // Ensure ID doesn't change
  };

  res.status(200).json({
    success: true,
    data: users[index]
  });
});

// @desc    Delete user
// @route   DELETE /api/users/:id
// @access  Private (we'll simulate this)
exports.deleteUser = asyncHandler(async (req, res, next) => {
  const user = users.find(user => user.id === req.params.id);

  if (!user) {
    return next(
      new ErrorResponse(`User not found with id of ${req.params.id}`, 404)
    );
  }

  // Delete user
  const index = users.findIndex(user => user.id === req.params.id);

  users.splice(index, 1);

  res.status(200).json({
    success: true,
    data: {}
  });
});
```

STEP 10: Create a new file in controllers → postController.js:

```
const ErrorResponse = require('../utils/errorResponse');
const asyncHandler = require('../middleware/async');
const { posts, users } = require('../utils/mockData');

// @desc    Get all posts
// @route   GET /api/posts
// @access  Public
exports.getPosts = asyncHandler(async (req, res, next) => {
  const page = parseInt(req.query.page, 10) || 1;
  const limit = parseInt(req.query.limit, 10) || 10;
  const startIndex = (page - 1) * limit;
  const endIndex = page * limit;
  const total = posts.length;

  const results = posts.slice(startIndex, endIndex);

  const enhancedResults = results.map(post => {
    const user = users.find(user => user.id === post.user_id);
    return {
      ...post,
      user: {
        id: user.id,
        username: user.username,
        full_name: user.full_name,
        profile_picture: user.profile_picture
      }
    };
  });

  const pagination = {};

  if (endIndex < total) {
    pagination.next = { page: page + 1, limit };
  }

  if (startIndex > 0) {
    pagination.prev = { page: page - 1, limit };
  }

  res.status(200).json({
    success: true,
    count: enhancedResults.length,
    page,
    total_pages: Math.ceil(total / limit),
    pagination,
    data: enhancedResults
```

```
exports.getPost = asyncHandler(async (req, res, next) => {
  const post = posts.find(post => post.id === req.params.id);

  if (!post) {
    return next(
      new ErrorResponse(`Post not found with id of ${req.params.id}`, 404)
    );
  }

  const user = users.find(user => user.id === post.user_id);
  const enhancedPost = {
    ...post,
    user: {
      id: user.id,
      username: user.username,
      full_name: user.full_name,
      profile_picture: user.profile_picture
    }
  };

  res.status(200).json({
    success: true,
    data: enhancedPost
  });
});

// @desc    Create new post
// @route   POST /api/posts
// @access  Private
exports.createPost = asyncHandler(async (req, res, next) => {
  const userId = req.header('X-User-Id');
  if (!userId) {
    return next(new ErrorResponse('Not authorized to access this route', 401));
  }

  const user = users.find(user => user.id === userId);
  if (!user) {
    return next(new ErrorResponse('User not found', 404));
  }

  const newPost = {
    id: (posts.length + 1).toString(),
    caption: req.body.caption,
    image: req.body.image,
    user_id: userId,
    created_at: new Date().toISOString().slice(0, 10)
```

```
  };

  posts.push(newPost);

  res.status(201).json({
    success: true,
    data: newPost
  });
});

// @desc    Update post
// @route   PUT /api/posts/:id
// @access  Private
exports.updatePost = asyncHandler(async (req, res, next) => {
  const userId = req.header('X-User-Id');
  if (!userId) {
    return next(new ErrorResponse('Not authorized to access this route', 401));
  }

  let post = posts.find(post => post.id === req.params.id);

  if (!post) {
    return next(
      new ErrorResponse(`Post not found with id of ${req.params.id}`, 404)
    );
  }

  if (post.user_id !== userId) {
    return next(new ErrorResponse('Not authorized to update this post', 401));
  }

  const index = posts.findIndex(post => post.id === req.params.id);
  posts[index] = {
    ...post,
    ...req.body,
    id: post.id,
    user_id: post.user_id
  };

  res.status(200).json({
    success: true,
    data: posts[index]
  });
});
```

STEP 11: Create a new file in routes → users.js:

```
const express = require('express');
const {
  getUsers, getUser, createUser, updateUser, deleteUser
} = require('../controllers/userController');

const router = express.Router();

router.route('/')
  .get(getUsers)
  .post(createUser);

router.route('/:id')
  .get(getUser)
  .put(updateUser)
  .delete(deleteUser);

module.exports = router;
```

STEP 12: Create a new file in routes → posts.js:

```
const express = require('express');
const {
  getPosts, getPost, createPost, updatePost, deletePost
} = require('../controllers/postController');

const router = express.Router();

router.route('/')
  .get(getPosts)
  .post(createPost);

router.route('/:id')
  .get(getPost)
  .put(updatePost)
  .delete(deletePost);

module.exports = router;
```

STEP 13: Create a new file in routes → likes.js:

```
const express = require('express');
const {
    getLikes,
    getLike,
    createLike,
    updateLike,
    deleteLike
} = require('../controllers/likeController');

const router = express.Router();

router.route('/').get(getLikes).post(createLike);

router.route('/:id').get(getLike).put(updateLike).delete(deleteLike);

module.exports = router;
```

STEP 14: Create a new file in routes → followers.js:

```
const express = require('express');
const {
    getFollowers,
    getFollower,
    createFollower,
    updateFollower,
    deleteFollower
} = require('../controllers/followerController');

const router = express.Router();

router.route('/').get(getFollowers).post(createFollower);

router.route('/:id').get(getFollower).put(updateFollower).delete(deleteFollower);

module.exports = router;
```

STEP 15: Create a new file in routes → comments.js:

```
const express = require('express');
const {
    getComments,
    getComment,
    createComment,
    updateComment,
    deleteComment
} = require('../controllers/commentController');

const router = express.Router();

router.route('/').get(getComments).post(createComment);

router.route('/:id').get(getComment).put(updateComment).delete(deleteComment);

module.exports = router;
```

STEP 16: Create a new file in **middleware → formatResponse.js:**

```
const formatResponse = (req, res, next) => {
  const originalJson = res.json;

  res.json = function(obj) {
    const acceptHeader = req.headers.accept;

    if (acceptHeader && acceptHeader.includes('application/xml')) {
      const convertToXml = (obj) => {
        let xml = '<?xml version="1.0" encoding="UTF-8"?>\n<response>\n';

        for (const key in obj) {
          if (Array.isArray(obj[key])) {
            xml += `  <${key}>\n`;
            obj[key].forEach(item => {
              xml += '    <item>\n';
              for (const itemKey in item) {
                xml += `      <${itemKey}>${item[itemKey]}</${itemKey}>\n`;
              }
              xml += '    </item>\n';
            });
```

```
            xml += `  </${key}>\n`;
          } else if (typeof obj[key] === 'object' && obj[key] !== null) {
            xml += `  <${key}>\n`;
            for (const nestedKey in obj[key]) {
              xml += `    <${nestedKey}>${obj[key][nestedKey]}</${nestedKey}>\n`;
            }
            xml += `  </${key}>\n`;
          } else {
            xml += `  <${key}>${obj[key]}</${key}>\n`;
          }
        }

        xml += '</response>';
        return xml;
      };

      res.set('Content-Type', 'application/xml');
      return res.send(convertToXml(obj));
    } else {
      res.set('Content-Type', 'application/json');
      return originalJson.call(this, obj);
    }
  };

  next();
};

module.exports = formatResponse;
```

Then add this middleware to our **server.js**:

```
app.use(require('./middleware/formatResponse'));
```

STEP 17: Create a simple HTML documentation page. Create a public folder and add a docs.html file:

- **mkdir public**
- **New-Item public/docs.html**

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Social Media API Documentation</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            margin: 0;
            padding: 20px;
            color: #333;
        }

        h1,
        h2,
        h3 {
            color: #0066cc;
        }

        .endpoint {
            margin-bottom: 30px;
            border-bottom: 1px solid #eee;
            padding-bottom: 20px;
        }

        .method {
            display: inline-block;
            padding: 5px 10px;
            border-radius: 5px;
            color: white;
            font-weight: bold;
        }

        .get {
            background-color: #61affe;
        }

        .post {
            background-color: #49cc90;
        }

        .put {
            background-color: #fca130;
        }

        .delete {
            background-color: #f93e3e;
        }

        .url {
            margin-left: 10px;
            font-family: monospace;
            font-size: 1.1em;
        }
```

```html
<html lang="en">
<head>
    <style>
        pre {
            background-color: #f5f5f5;
            padding: 10px;
            border-radius: 5px;
            overflow-x: auto;
        }

        table {
            border-collapse: collapse;
            width: 100%;
        }

        th,
        td {
            border: 1px solid #ddd;
            padding: 8px;
            text-align: left;
        }

        th {
            background-color: #f2f2f2;
        }
    </style>
</head>

<body>
    <h1>Social Media API Documentation</h1>
    <p>This is the documentation for the Social Media API, a RESTful API for a platform.</p>

    <h2>Users</h2>

    <div class="endpoint">
        <span class="method get">GET</span>
        <span class="url">/api/users</span>
        <p>Get a list of all users with pagination</p>

        <h3>Query Parameters</h3>
        <table>
            <tr>
                <th>Parameter</th>
                <th>Type</th>
                <th>Description</th>
            </tr>
            <tr>
                <td>page</td>
                <td>Number</td>
                <td>Page number (default: 1)</td>
            </tr>
            <tr>
                <td>limit</td>
                <td>Number</td>
                <td>Number of users per page (default: 10)</td>
            </tr>
        </table>
```

```
<html lang="en">
<body>
    <div class="endpoint">

        <h3>Response</h3>
        <pre>
{
  "success": true,
  "count": 2,
  "page": 1,
  "total_pages": 2,
  "pagination": {
    "next": {
      "page": 2,
      "limit": 10
    }
  },
  "data": [
    {
      "id": "1",
      "username": "traveler",
      "full_name": "Karma",
      "profile_picture": "https://example.com/profiles/traveler.jpg",
      "bio": "Travel photographer",
      "created_at": "2023-01-15"
    },
    ...
  ]
}
    </pre>
    </div>

    <div class="endpoint">
        <span class="method get">GET</span>
        <span class="url">/api/users/:id</span>
        <p>Get a specific user by ID</p>

        <h3>Parameters</h3>
        <table>
            <tr>
                <th>Parameter</th>
                <th>Type</th>
                <th>Description</th>
            </tr>
            <tr>
                <td>id</td>
                <td>String</td>
                <td>User ID</td>
            </tr>
        </table>

        <h3>Response</h3>
        <pre>
{
  "success": true,
  "data": {
    "id": "1",
    "username": "traveler",
```

```
<html lang="en">
<body>
    <div class="endpoint">
        <pre>
    "full_name": "Karma",
    "profile_picture": "https://example.com/profiles/traveler.jpg",
    "bio": "Travel photographer",
    "created_at": "2023-01-15"
  }
}
    </pre>
    </div>

    <div class="endpoint">
        <span class="method post">POST</span>
        <span class="url">/api/users</span>
        <p>Create a new user</p>

        <h3>Request Body</h3>
        <pre>
{
  "username": "new_traveler",
  "email": "new@example.com",
  "password": "securepassword",
  "full_name": "New Traveler",
  "bio": "Adventure seeker"
}
    </pre>

        <h3>Response</h3>
        <pre>
{
  "success": true,
  "data": {
    "id": "4",
    "username": "new_traveler",
    "full_name": "New Traveler",
    "email": "new@example.com",
    "profile_picture": "default-profile.jpg",
    "bio": "Adventure seeker",
    "created_at": "2023-03-20"
  }
}
    </pre>
    </div>


</body>

</html>
```

```
<div class="endpoint">
    <span class="method put">PUT</span>
    <span class="url">/api/users/:id</span>
    <p>Update an existing user</p>

    <h3>Parameters</h3>
    <table>
        <tr>
            <th>Parameter</th>
            <th>Type</th>
            <th>Description</th>
        </tr>
        <tr>
            <td>id</td>
            <td>String</td>
            <td>User ID</td>
        </tr>
    </table>

    <h3>Request Body</h3>
    <pre>
{
  "username": "updated_user",
  "full_name": "Updated Name",
  "bio": "Updated bio information"
}
    </pre>

    <h3>Response</h3>
    <pre>
{
  "success": true,
  "data": {
    "id": "1",
    "username": "updated_user",
    "full_name": "Updated Name",
    "bio": "Updated bio information",
    "updated_at": "2023-03-25"
  }
}
    </pre>
</div>

<!-- DELETE: Delete user -->
<div class="endpoint">
    <span class="method delete">DELETE</span>
    <span class="url">/api/users/:id</span>
    <p>Delete a user</p>

    <h3>Parameters</h3>
    <table>
        <tr>
            <th>Parameter</th>
            <th>Type</th>
            <th>Description</th>
        </tr>
        <tr>
```

```
                <td>id</td>
                <td>String</td>
                <td>User ID</td>
            </tr>
        </table>

        <h3>Response</h3>
        <pre>
{
  "success": true,
  "message": "User deleted successfully"
}
        </pre>
    </div>
    <h2>Posts</h2>

    <div class="endpoint">
        <span class="method get">GET</span>
        <span class="url">/api/posts</span>
        <p>Get all posts</p>
    </div>

    <div class="endpoint">
        <span class="method get">GET</span>
        <span class="url">/api/posts/:id</span>
        <p>Get a specific post by ID</p>
    </div>

    <div class="endpoint">
        <span class="method post">POST</span>
        <span class="url">/api/posts</span>
        <p>Create a new post</p>

        <h3>Request Body</h3>
        <pre>
{
  "user_id": "1",
  "content": "This is my first post!",
  "image": "https://example.com/post.jpg"
}
        </pre>
    </div>

    <div class="endpoint">
        <span class="method put">PUT</span>
        <span class="url">/api/posts/:id</span>
        <p>Update a post</p>
    </div>

    <div class="endpoint">
        <span class="method delete">DELETE</span>
        <span class="url">/api/posts/:id</span>
        <p>Delete a post</p>
```

```
<div class="endpoint">
</div>


<h2>Comments</h2>

<div class="endpoint">
    <span class="method get">GET</span>
    <span class="url">/api/posts/:postId/comments</span>
    <p>Get comments for a post</p>
</div>

<div class="endpoint">
    <span class="method post">POST</span>
    <span class="url">/api/comments</span>
    <p>Add a comment</p>

    <h3>Request Body</h3>
    <pre>
{
  "post_id": "1",
  "user_id": "2",
  "content": "Nice post!"
}
    </pre>
</div>

<div class="endpoint">
    <span class="method delete">DELETE</span>
    <span class="url">/api/comments/:id</span>
    <p>Delete a comment</p>
</div>


<h2>Likes</h2>

<div class="endpoint">
    <span class="method post">POST</span>
    <span class="url">/api/posts/:id/like</span>
    <p>Like a post</p>
</div>

<div class="endpoint">
    <span class="method delete">DELETE</span>
    <span class="url">/api/posts/:id/unlike</span>
    <p>Unlike a post</p>
</div>


<h2>Followers</h2>

<div class="endpoint">
    <span class="method post">POST</span>
    <span class="url">/api/users/:id/follow</span>
    <p>Follow a user</p>
</div>

<div class="endpoint">
```

```
    <span class="method delete">DELETE</span>
    <span class="url">/api/users/:id/unfollow</span>
    <p>Unfollow a user</p>
</div>

<div class="endpoint">
    <span class="method get">GET</span>
    <span class="url">/api/users/:id/followers</span>
    <p>Get user followers</p>
</div>

<div class="endpoint">
    <span class="method get">GET</span>
    <span class="url">/api/users/:id/following</span>
    <p>Get users being followed</p>
</div>
```

Step 18: Start your server:

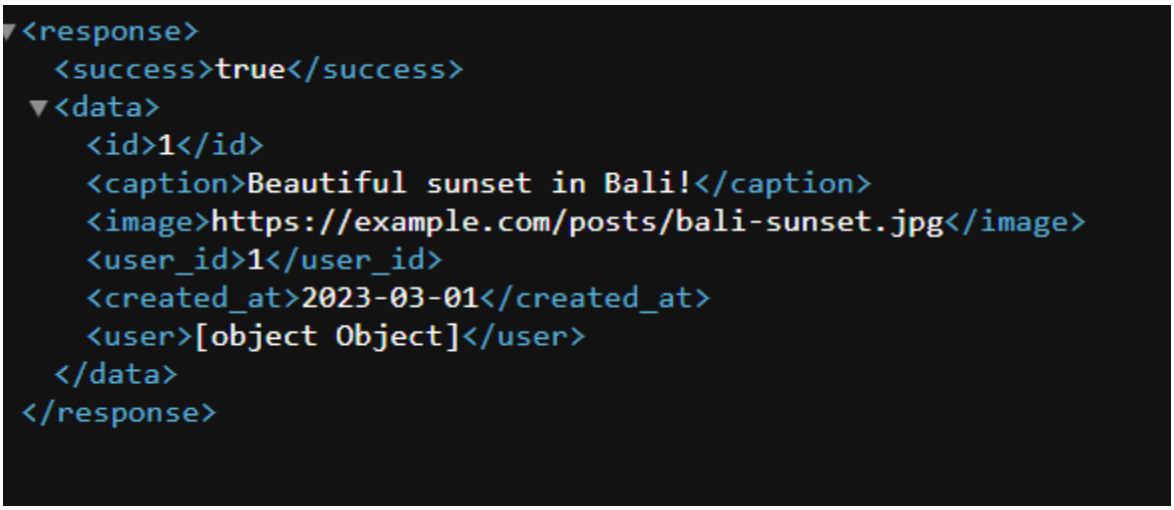
- npm run dev

- **curl -X GET http://localhost:3000/api/posts**

-

```
<response>
  <success>true</success>
  <count>4</count>
  <page>1</page>
  <total_pages>1</total_pages>
  <pagination> </pagination>
 <data>
  ▼<item>
    <id>1</id>
    <caption>Beautiful sunset in Bali!</caption>
    <image>https://example.com/posts/bali-sunset.jpg</image>
    <user_id>1</user_id>
    <created_at>2023-03-01</created_at>
    <user>[object Object]</user>
   </item>
  ▼<item>
    <id>2</id>
    <caption>Homemade pasta from scratch</caption>
    <image>https://example.com/posts/pasta.jpg</image>
    <user_id>2</user_id>
    <created_at>2023-03-05</created_at>
    <user>[object Object]</user>
   </item>
  ▼<item>
    <id>3</id>
    <caption>Morning workout complete!</caption>
    <image>https://example.com/posts/workout.jpg</image>
    <user_id>3</user_id>
    <created_at>2023-03-08</created_at>
    <user>[object Object]</user>
   </item>
  ▼<item>
    <id>4</id>
    <caption>Exploring the temples of Kyoto</caption>
    <image>https://example.com/posts/kyoto.jpg</image>
    <user_id>1</user_id>
    <created_at>2023-03-12</created_at>
    <user>[object Object]</user>
   </item>
  </data>
</response>
```

curl -X GET http://localhost:3000/api/posts/1



CONCLUSION:

This social media API lets users work with posts, users, comments, likes, and followers. Each part has clear actions: you can list all items, see one item, add new ones, update, or delete them. Private actions like creating or editing require the user's ID. The API sends structured responses with helpful info like totals and page numbers. Right now, the data comes from a mock file, but later it can use a real database. Overall, the design is organized, easy to understand, and ready to connect to a frontend.

REFERENCES:

- **Node.js. (2023).** ***Node.js v20.1.0 documentation*****.** **https://nodejs.org/en/docs/**

- **Express.js. (2023).** ***Express: Node.js web framework*****.** **https://expressjs.com/**

- **Mozilla Developer Network. (2023).** ***HTTP methods*****.** **https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods**

- **Postman. (2023).** ***Postman documentation*****.** **https://learning.postman.com/docs/**

- **W3Schools. (2023).** ***RESTful web services tutorial*****.** **https://www.w3schools.com/whatis/whatis_rest.asp**